

Database Systems Project
Review 2

Database Implementation and Core Functionalities

Gym Membership Management System (MongoDB)

Submitted by: Liz Alex (24BCE2978)
Malika Balaji (24BAI0114)
Nehaa Prabhu (24BAI0130)

Submitted to: Prof. Lydia Jane G

Contents

1. Review 2 Summary..... 3
1.1 Coverage of the Review 2 Rubric.....3
1.2 Expected Deliverables Checklist.....3
2. Implementation Environment..... 3
3. Database Implementation..... 4
3.1 Collections and Schema Validation.....4
3.2 Relationships Implemented.....5
3.3 Sample Data.....5
4. CRUD Operations..... 6
4.1 CRUD Coverage by Collection.....6
4.2 Referential Integrity Rules.....7
4.3 Key CRUD Implementations.....7
4.4 CRUD Demonstration Results.....8
5. Advanced NoSQL Features..... 10
5.1 Aggregation Pipelines.....10
5.2 Graph Traversal: Referral Network.....15
5.3 Indexing.....16
5.4 Multi-Document ACID Transactions.....17
6. Traceability to Review 1 Requirements..... 19
7. Testing..... 19
8. Intermediate Demonstration Plan..... 20
9. Source Code Repository..... 20
10. Plan for Review 3..... 21

1. Review 2 Summary

Review 1 defined the problem, requirements, NoSQL selection and the seven-collection MongoDB data model. Review 2 implements that design as a working database. The **gym_db** database now runs with schema validation, **1,016 sample documents**, a complete CRUD layer for every collection, and a set of advanced MongoDB features: eight aggregation pipelines, recursive graph traversal with `$graphLookup`, nineteen purpose-built indexes, and multi-document ACID transactions.

1.1 Coverage of the Review 2 Rubric

Criterion	Marks	What was delivered	Section
Database implementation (collections, relationships, sample data)	2	7 collections with $jsonSchema validators; 10 relationships using references and embedded arrays; 1,016 reproducible sample documents	3
CRUD operations completed	3	36 functions covering create, read, update and delete on all 7 collections, with 10 referential-integrity rules enforced	4
Advanced NoSQL features	5	8 aggregation pipelines and 1 view; $graphLookup referral network; 19 indexes (unique, compound, multikey, text, partial) with explain() proof; 2 ACID transactions with rollback	5
Total	**10**		

1.2 Expected Deliverables Checklist

Deliverable	Status	Evidence
Database created	Done	gym_db with 7 validated collections, 19 secondary indexes and 1 view (Section 3.1)
Sample data loaded	Done	seed.py loads 1,016 documents; also exported to data/*.json for mongoimport (Section 3.3)
CRUD operations working	Done	gym_db/crud.py, exercised by demo.py and 6 automated tests (Sections 4 and 7)
Minimum 50 sample records	Done	60 members alone; 1,016 documents in total
Intermediate demonstration	Ready	demo.py runs every rubric item in order; mongosh/demo_queries.js for live queries (Section 8)
Source code repository	Ready	gym-membership-db Git repository with README (Section 9)

2. Implementation Environment

All reports use a fixed reference date of **24 September 2026** (set in `gym_db/config.py`), and the seed script uses a fixed random seed. Every run therefore produces the same dataset and the same report figures, which keeps the demonstration repeatable.

Component	Choice	Reason
Database	MongoDB 8.3 (Homebrew, macOS), single-node replica set rs0	A replica set is required for multi-document transactions
Language and driver	Python 3.12 with PyMongo 4	Official driver; readable pipelines; easy to demonstrate
Password hashing	bcrypt	Salted adaptive hashing, as specified in NFR3
Shell	mongosh	Live ad-hoc queries during the review
Testing	pytest with mongomock (in-memory MongoDB)	Tests run on any laptop without a database server

3. Database Implementation

3.1 Collections and Schema Validation

MongoDB does not require a schema, but the project attaches a `$jsonSchema` validator to every collection with `validationLevel: strict` and `validationAction: error`. Invalid documents are rejected by the database itself, not only by application code. Each validator enforces required fields, BSON data types, allowed values, value ranges and formats.

Collection	Key rules enforced by the validator
members	member_id pattern MEM####; valid email; 10-digit phone; tier in Basic, Premium, VIP; status in active, expired, frozen; dates are BSON dates; trainer and referrer are ObjectIds
trainers	trainer_id pattern TRN##; role fixed to trainer; specialization is an array of strings; availability items need day, start and end time
workout_plans	member and trainer ObjectIds required; duration 1 to 52 weeks; at least one exercise; sets and reps at least 1
classes	trainer ObjectId required; capacity 1 to 50; schedule and roster are arrays
class_sessions	class ObjectId; date is a BSON date; status in scheduled, completed, cancelled
attendance	session and member ObjectIds; status in present, absent, late
payments	amount at least 0; currency INR; mode in cash, card, UPI; status in paid, pending, overdue

Validator for the members collection (gym_db/schema.py):

```
    "members": {
        "bsonType": "object",
        "required": ["member_id", "name", "email", "password_hash", "role", "membership"],
        "properties": {
            "member_id": {"bsonType": "string", "pattern": "^MEM[0-9]{4}$"},
            "name": {"bsonType": "string", "minLength": 2},
            "email": {"bsonType": "string", "pattern": "^.+@.+\\..+$"},
            "phone": {"bsonType": "string", "pattern": "^[0-9]{10}$"},
            "gender": {"enum": ["Male", "Female", "Other"]},
            "role": {"enum": ["member"]},
            "membership": {
                "bsonType": "object",
                "required": ["type", "status", "start_date", "end_date"],
                "properties": {
                    "type": {"enum": ["Basic", "Premium", "VIP"]},
                    "status": {"enum": ["active", "expired", "frozen"]},
                    "start_date": {"bsonType": "date"},
                    "end_date": {"bsonType": "date"},
                },
            },
            "assigned_trainer_id": {"bsonType": [OID, "null"]},
            "referred_by": {"bsonType": [OID, "null"]},
        },
    }
```

In the live run, inserting a member with the tier `Gold` was rejected by the server with error code 121, Document failed validation (Screenshot 1).

```
--- Relationship check: one member resolved across all referenced collections
MEM1001 Harish Kumar (VIP) -> trainer TRN01 Priya Menon
  workout plans: 1 | classes: 1 | attendance rows: 5 | payments: 4

--- Validator in action: inserting a member with an invalid tier
REJECTED by validator: 121 Document failed validation
```

Screenshot 1: $jsonSchema validator rejecting an invalid membership tier (live MongoDB run)

3.2 Relationships Implemented

The relationships from the Review 1 data model are implemented as designed: bounded data that is always read with its parent is **embedded**; data that grows without limit or is queried independently is **referenced** by ObjectId, with frequently displayed names **denormalized** to avoid joins on dashboards.

Relationship	Cardinality	Implementation	Integrity enforced by
Trainer to Members	1:N	members.assigned_trainer_id reference	create_member, assign_trainer
Member to Workout plans	1:N	workout_plans.member_id reference + member_name copy	create_workout_plan
Trainer to Workout plans	1:N	workout_plans.trainer_id reference + trainer_name copy	create_workout_plan
Plan to Exercises	1:N	exercises[ ] embedded array	validator (minItems 1)
Trainer to Classes	1:N	classes.trainer_id reference	create_class, delete_trainer
Class to Members (roster)	M:N	enrolled_members[ ] embedded, bounded by capacity	enroll_member (atomic)
Class to Sessions	1:N	class_sessions.class_id reference	create_session, delete_class
Session and Member to Attendance	1:N	attendance references both, unique pair	unique index + upsert
Member to Payments	1:N	payments.member_id reference + member_name copy	create_invoice, delete_member
Member to Member (referral)	1:N, recursive	members.referred_by self-reference (new)	create_member, delete_member

Change from Review 1. One nullable field, `referred_by`, was added to members to model the gym's referral programme. It makes the members collection a self-referencing graph and gives the graph traversal feature (Section 5.2) a real business purpose. No other schema changes were needed.

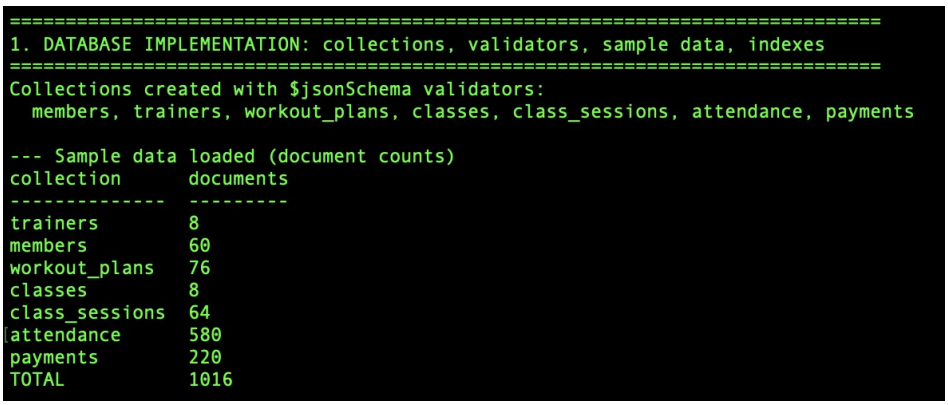
3.3 Sample Data

The seed script (`gym_db/seed.py`) generates realistic, internally consistent data: every reference points to an existing document, rosters never exceed capacity, attendance exists only for completed sessions of enrolled members, and each bill matches the member's tier fee (Basic INR 1,500, Premium INR 2,500, VIP INR 4,000 per month).

Collection	Documents	Contents
trainers	8	Specializations, certifications and weekly availability
members	60	24 Basic, 22 Premium, 14 VIP; 50 active, 6 expired, 4 frozen; 5 cities; referral links
workout_plans	76	60 active and 16 completed plans, 3 to 4 embedded exercises each
classes	8	Yoga, Zumba, CrossFit, HIIT, Spin, Pilates, Strength; 2 weekly slots each
class_sessions	64	Four weeks of dated sessions, 7 September to 4 October 2026
attendance	580	Present, late or absent for each enrolled member in each completed session

Collection	Documents	Contents
payments	220	July to October 2026 billing cycles: paid, pending and overdue
Total	**1,016**	The rubric minimum is 50

Passwords are stored as bcrypt hashes. Test logins follow the pattern `Trainer@01` for TRN01 and `Member@1001` for MEM1001. The full dataset is also exported as MongoDB Extended JSON in the repository's `data/` folder and can be loaded with `mongoimport --jsonArray`.



Screenshot 2: Collections created and 1,016 sample documents loaded (live MongoDB run)

4. CRUD Operations

All CRUD logic is in `gym_db/crud.py`: **36 functions** across the seven collections. Because MongoDB has no foreign keys, this layer is also where referential integrity is enforced, as committed in NFR5 of Review 1.

4.1 CRUD Coverage by Collection

Collection	Create	Read	Update	Delete
members	create_member	get_member, list_members (filter, sort, paginate)	update_member, change_membership, assign_trainer	delete_member (guarded cascade)
trainers	create_trainer	get_trainer, find_trainers_by_skill	add_specialization ($addToSet), set_availability	delete_trainer (guarded)
workout_plans	create_workout_plan	get_active_plan	add_exercise ($push), progress_exercise (positional $inc)	remove_exercise ($pull), delete_workout_plan
classes	create_class	get_class, member_classes (multikey)	enroll_member (atomic), withdraw_member	delete_class
class_sessions	create_session	used by reports and T2	cancel_session	via delete_class
attendance	mark_attendance (upsert)	attendance_history (sorted, indexed)	mark_attendance (corrects)	delete_attendance
payments	create_invoice	member_dues	record_payment, flag_overdue (bulk)	delete_payment (pending only)

Authentication (FR8) is provided by `authenticate(email, password)`, which verifies the bcrypt hash and returns the user's role. Every read that returns a member or trainer excludes `password_hash` through a projection.

4.2 Referential Integrity Rules

Rule	Enforced in
A member's trainer, and a referrer, must exist	create_member, assign_trainer
A workout plan is issued by the member's assigned trainer; only one plan is active at a time	create_workout_plan
Only active members can enroll; roster never exceeds capacity; no duplicate enrollment	enroll_member (single atomic update)
A name change is copied to every denormalized copy (plans, attendance, payments, class rosters)	update_member
A member with unpaid bills cannot be deleted; otherwise plans, roster entries and referral links are cleaned up	delete_member
A trainer with assigned members or classes cannot be deleted	delete_trainer
Deleting a class removes its future sessions but keeps attendance history	delete_class
Paid bills are immutable; only a pending invoice raised in error can be deleted	record_payment, delete_payment
One attendance record per member per session	unique index uq_session_member + upsert
Pending bills past their due date become overdue	flag_overdue (bulk update)

4.3 Key CRUD Implementations

Atomic class enrollment (Update)

The capacity and duplicate checks sit inside the update filter using `$expr` and `$size`. MongoDB applies a single-document update atomically, so two members booking the last place at the same moment can never overbook the class, and no transaction is needed.

```
def enroll_member(db, class_id, member_id):
    """FR4. Atomic single-document update: the filter itself checks capacity and duplicates,
    so two simultaneous requests can never overbook the class."""
    m = db.members.find_one({"member_id": member_id, "membership.status": "active"})
    if not m:
        raise IntegrityError(f"{member_id} not found or membership not active")
    res = db.classes.update_one(
        {"class_id": class_id,
         "enrolled_members.member_id": {"$ne": m["_id"]},
         "$expr": {"$lt": [{"$size": "$enrolled_members"}, "$capacity"]}},
        {"$push": {"enrolled_members": {"member_id": m["_id"], "member_name": m["name"],
                                        "enrolled_on": REFERENCE_DATE}}})
    if res.modified_count == 0:
        raise IntegrityError(f"Cannot enroll {member_id} in {class_id}: class full or already enrolled")
    return True
```

Propagating a name change to denormalized copies (Update)

```
def update_member(db, member_id, **fields):
    """Update profile fields. A name change is propagated to every denormalized copy."""
    allowed = {"name", "phone", "email", "address", "emergency_contact"}
    bad = set(fields) - allowed
    if bad:
        raise ValueError(f"Cannot update {bad} here")
    doc = db.members.find_one_and_update({"member_id": member_id}, {"$set": fields},
                                         projection=SAFE, return_document=ReturnDocument.AFTER)
    if doc and "name" in fields:
        oid, name = doc["_id"], fields["name"]
        for coll in ("workout_plans", "attendance", "payments"):
            db[coll].update_many({"member_id": oid}, {"$set": {"member_name": name}})
        db.classes.update_many({"enrolled_members.member_id": oid},
                               {"$set": {"enrolled_members.$.member_name": name}})  # positional $
```

```
    return doc
```

Guarded cascading delete (Delete)

```
def delete_member(db, member_id):
    """Blocked if dues are outstanding; otherwise cascades to plans and class rosters.
    Attendance and payment history are kept as the gym's audit trail."""
    oid = _member_oid(db, member_id)
    dues = db.payments.count_documents({"member_id": oid, "status": {"$in": ["pending", "overdue"]}})
    if dues:
        raise IntegrityError(f"{member_id} has {dues} unpaid bill(s); settle before deleting")
    plans = db.workout_plans.delete_many({"member_id": oid}).deleted_count
    rosters = db.classes.update_many({}, {"$pull": {"enrolled_members": {"member_id":
oid}}}).modified_count
    db.members.update_many({"referred_by": oid}, {"$set": {"referred_by": None}})
    db.members.delete_one({"_id": oid})
    return {"plans_deleted": plans, "classes_updated": rosters}
```

Idempotent attendance marking with upsert (Create and Update)

```
def mark_attendance(db, session_id, member_id, status, check_in_time=None):
    """FR5. Upsert keyed on (session, member): re-marking corrects the record, never duplicates it."""
    s = db.class_sessions.find_one({"session_id": session_id})
    if not s:
        raise IntegrityError(f"Session {session_id} not found")
    m = db.members.find_one({"member_id": member_id})
    if not m:
        raise IntegrityError(f"Member {member_id} not found")
    return db.attendance.update_one(
        {"session_id": s["_id"], "member_id": m["_id"]},
        {"$set": {"status": status, "check_in_time": check_in_time},
         "$setOnInsert": {"class_id": s["class_id"], "member_name": m["name"], "date": s["date"]}},
        upsert=True)
```

4.4 CRUD Demonstration Results

Results recorded from the live run of `python demo.py` (Screenshots 3 and 4):

Operation	Result
Create trainer, member, plan, class, session, invoice	TRN09, MEM1061 (referred by MEM1003), WP5077, CLS09 (capacity 2), SES9065, PAY7221
Create with a duplicate email	Rejected: DuplicateKeyError from unique index uq_email
Create a member for trainer TRN77	Rejected: IntegrityError, Trainer TRN77 not found
Read active VIP members, page 1	11 matches; first 5 returned, sorted by member_id
Read trainers with specialization Yoga	TRN03 Deepa Krishnan, TRN07 Kavitha Reddy
Enroll 3 members into a class of capacity 2	First two succeed; third rejected, class full
Push exercise, +5 kg on Squat, add a skill twice	Plan shows 3 exercises, Squat 30 to 35 kg; skill stored once ($addToSet)
Rename member	New name appears in members, workout_plans, payments and the class roster
Mark attendance late, then correct to present	Still exactly 1 attendance row, status present
Record payment; run overdue sweep	PAY7221 paid by UPI; 16 bills flagged overdue
Delete a member with unpaid bills	Blocked: MEM1008 has 2 unpaid bills
Delete MEM1061	Member deleted, 1 plan cascaded, 0 orphan documents left
Delete trainer TRN01	Blocked: still has 6 members; reassign first
Login with correct and wrong password	Returns role trainer for TRN01; wrong password returns None

```
============================================================================
2. CRUD OPERATIONS
============================================================================

--- CREATE: new trainer, new member (referred by MEM1003), workout plan, class, session, invoice
trainer TRN09 | member MEM1061 | plan WP5077 | class CLS09 | session SES9065 | invoice PAY7221

--- CREATE rejected: duplicate email (unique index uq_email)
DuplicateKeyError: email already registered

--- CREATE rejected: member assigned to a trainer that does not exist (app-level integrity)
IntegrityError: Trainer TRN77 not found

--- READ: member profile MEM1061 (password_hash excluded by projection)
{'_id': ObjectId('6ab528fd6d2803a432e142d5'),
 'member_id': 'MEM1061',
 'name': 'Rhea Thomas',
 'email': 'rhea.thomas@example.com',
 'phone': '9876501234',
 'dob': datetime.datetime(2004, 3, 15, 0, 0),
 'gender': 'Female',
 'role': 'member',
 'membership': {'type': 'Premium',
                'status': 'active',
                'start_date': datetime.datetime(2026, 9, 1, 0, 0),
                'end_date': datetime.datetime(2027, 9, 1, 0, 0)},
 'assigned_trainer_id': ObjectId('6ab528fd6d2803a432e142d4'),
 'referred_by': ObjectId('6ab528f96d2803a432e13ee5'),
 'address': {'line1': '', 'city': 'Vellore', 'pincode': '632014'},
 'emergency_contact': {'name': '', 'phone': ''},
 'created_at': datetime.datetime(2026, 9, 24, 0, 0)}

--- READ: VIP members, page 1 (filter + sort + skip/limit pagination)
11 active VIP members; first 5:
member_id  name              city
---------  ----------------  ---------
MEM1001    Harish Kumar      Bengaluru
MEM1012    Isha Das          Vellore
MEM1014    Divya Das         Chennai
MEM1018    Aishwarya Das     Hosur
MEM1028    Lakshmi Krishnan  Bengaluru

--- READ: trainers who teach Yoga (query inside embedded array)
trainer_id  name
----------  --------------
TRN03       Deepa Krishnan
TRN07       Kavitha Reddy

--- READ: MEM1001 last 5 attendance records (uses index ix_member_date)
date                status   check_in_time
------------------  -------  -------------
2026-09-23 00:00:00  present  18:52
2026-09-19 00:00:00  present  16:51
2026-09-16 00:00:00  present  18:50
2026-09-12 00:00:00  present  16:57
2026-09-09 00:00:00  present  19:00
```

Screenshot 3: CRUD demonstration, create and read operations (live MongoDB run)

```
--- READ: MEM1001 last 5 attendance records (uses index ix_member_date)
date                status   check_in_time
------------------  -------  -------------
2026-09-23 00:00:00  present  18:52
2026-09-19 00:00:00  present  16:51
2026-09-16 00:00:00  present  18:50
2026-09-12 00:00:00  present  16:57
2026-09-09 00:00:00  present  19:00

--- UPDATE: enroll into a class of capacity 2 (atomic capacity guard)
3rd enrollment blocked -> Cannot enroll MEM1001 in CLS09: class full or already enrolled
roster: ['Rhea Thomas', 'Swathi Sharma']

--- UPDATE: embedded arrays ($push, positional $inc, $addToSet, $pull)
day  exercise_name  sets  reps  weight_kg
---  -------------  ----  ----  ---------
Mon  Squat          4     10    35
Wed  Bench Press    4     10    25
Fri  Deadlift       3     8     40
specialization: ['Kickboxing', 'HIIT', 'Boxing']

--- UPDATE: tier upgrade Premium -> VIP, and name change propagated to denormalized copies
members: Rhea T. Thomas | tier: VIP
workout_plans.member_name: Rhea T. Thomas
payments.member_name: Rhea T. Thomas
classes.enrolled_members.member_name: ['Rhea T. Thomas']

--- UPDATE: attendance upsert (mark late, then correct to present; still one row)
rows for (SES9065, MEM1061): 1 | status: present

--- UPDATE: payments (record payment, overdue sweep)
record_payment: {'payment_id': 'PAY7221', 'amount': 2500, 'payment_mode': 'UPI', 'status': 'paid'}
bills newly flagged overdue by sweep: 16

--- DELETE: remove exercise, withdraw from class, delete attendance, delete member with cascade
exercises now: ['Squat', 'Bench Press']
withdraw: 1 | delete attendance: 1
delete MEM1016 blocked -> MEM1016 has 2 unpaid bill(s); settle before deleting
delete MEM1061 -> {'plans_deleted': 1, 'classes_updated': 0}
member still exists? False | orphan plans left: 0
delete TRN01 blocked -> TRN01 still has 6 members and 0 classes; reassign first

--- AUTH (FR8): bcrypt login check
correct password: {'role': 'trainer', 'id': 'TRN01', 'name': 'Priya Menon'}
wrong password:   None
```

Screenshot 4: CRUD demonstration, update, delete and authentication (live MongoDB run)

5. Advanced NoSQL Features

This criterion carries 5 of the 10 marks. Four categories of advanced features are implemented, each tied to a requirement from Review 1.

Feature	Implemented	Requirement
Aggregation framework	8 analytical pipelines and 1 read-only view	FR2, FR3, FR6, FR7
Graph traversal	$graphLookup over the member referral network (2 queries)	Referral programme
Indexing	19 indexes of 6 kinds, verified with explain()	NFR1
Transactions	2 multi-document ACID transactions with rollback	NFR5
Also used	$jsonSchema validation, atomic conditional updates, upserts, array update operators	NFR5

5.1 Aggregation Pipelines

ID	Report	Business question answered	Main stages and operators
A1	Revenue trend	How much was collected each month, per tier?	$match, $lookup, $unwind, $group (twice), $dateToString, $push
A2	Class popularity	Which classes are full, and which are actually attended?	$group, $cond, $addToSet, $lookup, $size, $divide, $round
A3	At-risk members	Who attends less than 60% and may cancel?	$group, $addFields, $match, $lookup
A4	Outstanding dues	Who owes money, how late, and in what priority?	$subtract on dates, $floor, $max, $group, nested $cond
A5	Trainer workload	Is the load spread fairly across trainers?	3 x $lookup, $filter, $size

ID	Report	Business question answered	Main stages and operators
A6	Membership overview	Tier, status, age and city mix in one query	$facet, $bucket, $group
A7	Top exercises	What do trainers prescribe most?	$unwind (embedded array), $group, $avg
A8	Peak hours	When is the gym busiest?	$substr, $concat, $group, $sort
V1	member_dashboard view	Everything a member sees on login	stored view: 2 x $lookup, $filter, $map, $first

A1: Monthly revenue by tier

```
def revenue_trend(db):
    return list(db.payments.aggregate([
        {"$match": {"status": "paid"}},
        {"$lookup": {"from": "members", "localField": "member_id", "foreignField": "_id", "as": "m"}},
        {"$unwind": "$m"},
        {"$group": {
            "_id": {"month": {"$dateToString": {"format": "%Y-%m", "date": "$billing_period_start"}},
                    "tier": "$m.membership.type"},
            "revenue": {"$sum": "$amount"}, "payments": {"$sum": 1}}},
        {"$group": {"_id": "$_id.month",
                    "total_revenue": {"$sum": "$revenue"},
                    "payments": {"$sum": "$payments"},
                    "by_tier": {"$push": {"tier": "$_id.tier", "revenue": "$revenue"}}}},
        {"$sort": {"_id": 1}},
        {"$project": {"_id": 0, "month": "$_id", "total_revenue": 1, "payments": 1, "by_tier": 1}},
    ]))
```

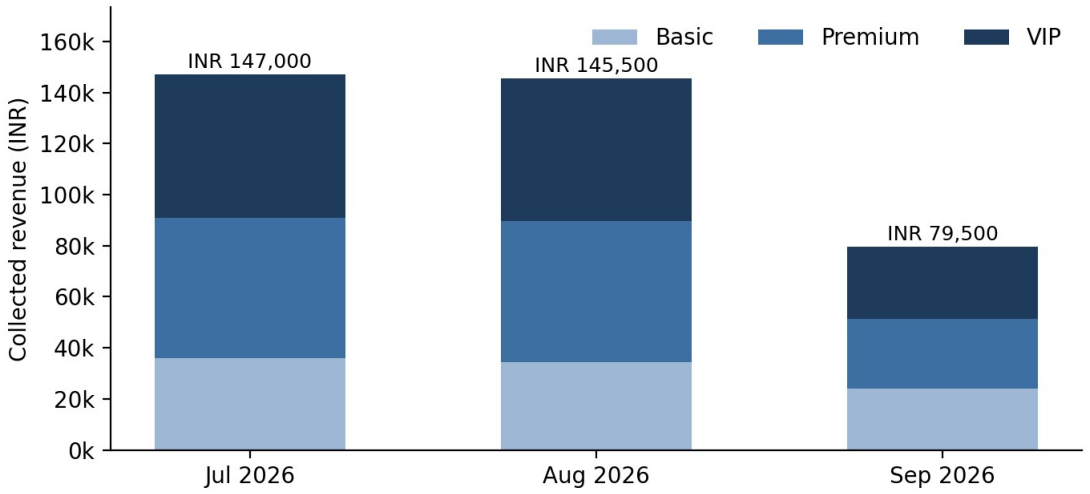


Figure 1: Output of pipeline A1, collected revenue by month and membership tier

Revenue was INR 1,47,000 in July and INR 1,45,500 in August, then fell to INR 79,500 in September. Pipelines A6 and A4 explain the drop: 6 memberships expired on 31 August, 4 are frozen, and 16 September bills are unpaid. VIP members are 14 of 60 but produced INR 56,000 of July revenue, 38% of the total.

A2: Class popularity

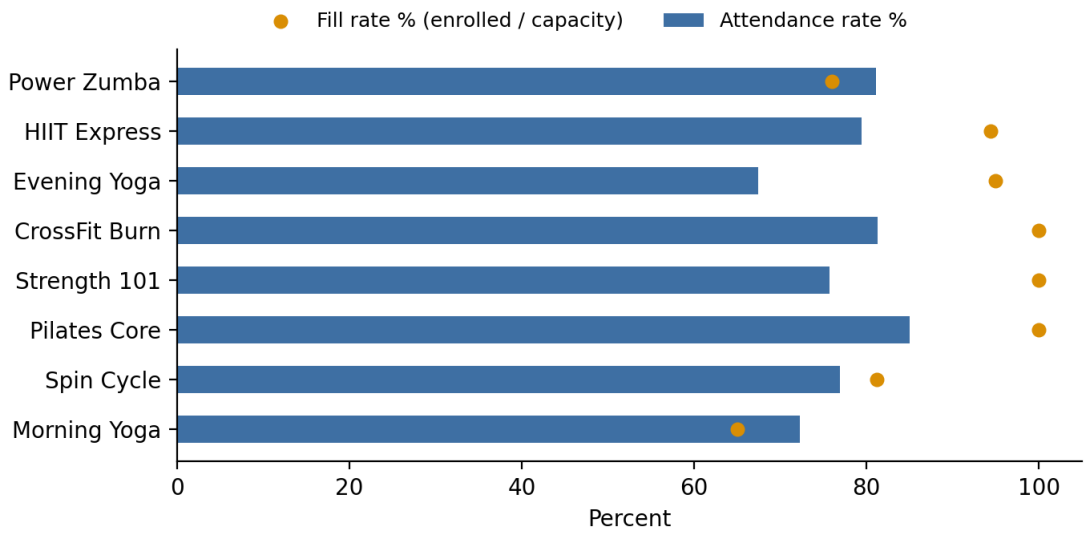


Figure 2: Output of pipeline A2, attendance rate and fill rate per class

Class	Trainer	Enrolled / capacity	Sessions held	Avg turnout	Attendance %
Power Zumba	Meenakshi Iyer	19 / 25	5	15.4	81.1
HIIT Express	Rohit Sharma	17 / 18	4	13.5	79.4
Evening Yoga	Kavitha Reddy	19 / 20	5	12.8	67.4
CrossFit Burn	Vivek Nair	15 / 15	5	12.2	81.3
Strength 101	Sanjay Pillai	14 / 14	5	10.6	75.7
Pilates Core	Deepa Krishnan	12 / 12	5	10.2	85
Spin Cycle	Farhan Ali	13 / 16	4	10	76.9
Morning Yoga	Deepa Krishnan	13 / 20	5	9.4	72.3

Pilates Core, CrossFit Burn and Strength 101 are fully booked, and Pilates Core has the best attendance at 85%. Morning Yoga is weakest on both measures (65% full, 72.3% attendance), which makes it a candidate for a new time slot.

A3 and A4: Members at risk and outstanding dues

Member	Name	Tier	Sessions	Attended	Rate %
MEM1052	Isha Nair	Premium	5	1	20
MEM1020	Keerthana Iyer	Premium	10	4	40
MEM1030	Meera Varma	Basic	24	11	45.8
MEM1008	Isha Joshi	Basic	10	5	50
MEM1019	Siddharth Varma	Basic	24	12	50
MEM1039	Gautham Joshi	Premium	14	7	50
MEM1053	Karthik Krishnan	Basic	8	4	50

A3 output: members attending less than 60% of their sessions

Member	Name	Tier	Bills	Amount due	Days overdue	Priority
MEM1016	Divya Menon	Basic	2	INR 3,000	50	HIGH

Member	Name	Tier	Bills	Amount due	Days overdue	Priority
MEM1014	Divya Das	VIP	2	INR 8,000	19	HIGH
MEM1018	Aishwarya Das	VIP	2	INR 8,000	19	HIGH
MEM1038	Nandini Das	VIP	2	INR 8,000	19	HIGH
MEM1046	Nandini Menon	VIP	2	INR 8,000	19	HIGH
MEM1009	Gautham Bhat	Premium	2	INR 5,000	19	HIGH

A4 output: first six rows of the collections worklist

A4 calculates days overdue from the dates inside the pipeline and then assigns a priority with nested `$cond`. MEM1016 holds the oldest debt at 50 days, with both August and September unpaid.

A5 to A8: Operational reports

Trainer	Name	Members	Active plans	Classes
TRN03	Deepa Krishnan	12	12	2
TRN04	Sanjay Pillai	10	10	1
TRN06	Farhan Ali	10	10	1
TRN08	Vivek Nair	8	8	1
TRN07	Kavitha Reddy	7	7	1
TRN01	Priya Menon	6	6	0
TRN02	Rohit Sharma	5	5	1
TRN05	Meenakshi Iyer	2	2	1

A5 output: trainer workload from three $lookup joins

A5 shows an uneven load: Deepa Krishnan handles 12 members and 2 classes, while Meenakshi Iyer has only 2 members. A6 returns four summaries from a single `$facet` query: by tier (Basic 24, Premium 22, VIP 14), by status (active 50, expired 6, frozen 4), by age band using `$bucket` (18 to 24: 12, 25 to 34: 15, 35 to 44: 22, 45 to 59: 11), and by city.

```
def membership_overview(db, today=REFERENCE_DATE):
    res = list(db.members.aggregate([
        {"$addFields": {"age": {"$floor": {"$divide": [{"$subtract": [today, "$dob"]}, 31557600000]}}}},
        {"$facet": {
            "by_tier": [{"$group": {"_id": "$membership.type", "count": {"$sum": 1}}}, {"$sort": {"_id":
1}}],
            "by_status": [{"$group": {"_id": "$membership.status", "count": {"$sum": 1}}}, {"$sort":
{"_id": 1}}],
            "age_groups": [{"$bucket": {"groupBy": "$age", "boundaries": [18, 25, 35, 45, 60],
                                        "default": "60+", "output": {"count": {"$sum": 1}}}}],
            "by_city": [{"$group": {"_id": "$address.city", "count": {"$sum": 1}}}, {"$sort": {"count": -
1}}],
        }},
    ]))
    return res[0]
```

A7 unwinds the embedded exercise arrays of all active plans. The most prescribed exercises are Plank (23 plans), Rowing Machine (20 plans), Squat (19 plans, average 67.6 kg), Bench Press (17 plans, average 55.9 kg), Deadlift (17 plans, average 74.4 kg). A8 finds the busiest check-in hours: 18:00 (115 check-ins), 06:00 (98 check-ins), 17:00 (82 check-ins).

V1 is stored in the database as a read-only view named `member_dashboard`. The member screen in Review 3 will query this view directly; it joins the trainer's name and totals unpaid bills while never exposing the password hash.

```
======================================================================
3a. AGGREGATION PIPELINES (FR7)
======================================================================

--- A1 Monthly revenue trend by tier (paid only)
2026-07  INR  147,000  (60 payments)  [Basic 36,000, Premium 55,000, VIP 56,000]
2026-08  INR  145,500  (59 payments)  [Basic 34,500, Premium 55,000, VIP 56,000]
2026-09  INR   79,500  (34 payments)  [Basic 24,000, Premium 27,500, VIP 28,000]

--- A2 Class popularity
class_name     trainer         enrolled  capacity  fill_rate_pct  sessions_held  avg_turnout  attendance_rate_pct
-------------  --------------  --------  --------  -------------  -------------  -----------  -------------------
Power Zumba    Meenakshi Iyer  19        25        76.0           5              15.4         81.1
HIIT Express   Rohit Sharma    17        18        94.4           4              13.5         79.4
Evening Yoga   Kavitha Reddy   19        20        95.0           5              12.8         67.4
CrossFit Burn  Vivek Nair      15        15        100.0          5              12.2         81.3
Strength 101   Sanjay Pillai   14        14        100.0          5              10.6         75.7
Pilates Core   Deepa Krishnan  12        12        100.0          5              10.2         85.0
Spin Cycle     Farhan Ali      13        16        81.2           4              10.0         76.9
Morning Yoga   Deepa Krishnan  13        20        65.0           5              9.4          72.3

--- A3 Members at risk (attendance < 60%)
member_id  name              tier     sessions_marked  attended  rate_pct
---------  ----------------  -------  ---------------  --------  --------
MEM1052    Isha Nair         Premium  5                1         20.0
MEM1020    Keerthana Iyer    Premium  10               4         40.0
MEM1030    Meera Varma       Basic    24               11        45.8
MEM1008    Isha Joshi        Basic    10               5         50.0
MEM1019    Siddharth Varma   Basic    24               12        50.0
MEM1053    Karthik Krishnan  Basic    8                4         50.0
MEM1039    Gautham Joshi     Premium  14               7         50.0

--- A4 Outstanding dues (collections worklist)
member_id  name              tier     bills  amount_due  max_days_overdue  priority
---------  ----------------  -------  -----  ----------  ----------------  --------
MEM1016    Divya Menon       Basic    2      3000        50.0              HIGH
MEM1046    Nandini Menon     VIP      2      8000        19.0              HIGH
MEM1038    Nandini Das       VIP      2      8000        19.0              HIGH
MEM1018    Aishwarya Das     VIP      2      8000        19.0              HIGH
MEM1014    Divya Das         VIP      2      8000        19.0              HIGH
MEM1021    Karthik Sharma    Premium  2      5000        19.0              HIGH
MEM1054    Keerthana Krishnan  Premium  2      5000        19.0              HIGH
MEM1044    Kavya Kumar       Premium  2      5000        19.0              HIGH
MEM1052    Isha Nair         Premium  2      5000        19.0              HIGH
MEM1009    Gautham Bhat      Premium  2      5000        19.0              HIGH
MEM1024    Ananya Das        Premium  2      5000        19.0              HIGH
MEM1049    Ashwin Krishnan   Premium  2      5000        19.0              HIGH

--- A5 Trainer workload (3 x $lookup)
trainer_id  name            members  active_plans  classes
----------  --------------  -------  ------------  -------
TRN03       Deepa Krishnan  12       12            2
TRN04       Sanjay Pillai   10       10            1
TRN06       Farhan Ali      10       10            1
TRN08       Vivek Nair      8        8             1
TRN07       Kavitha Reddy   7        7             1
TRN01       Priya Menon     6        6             0
TRN02       Rohit Sharma    5        5             1
TRN05       Meenakshi Iyer  2        2             1
TRN09       Nisha Varghese  0        0             1
```

Screenshot 5: Aggregation pipelines A1 to A5 (live MongoDB run)

```
--- A6 Membership overview ($facet + $bucket in one query)
{'by_tier': [{'_id': 'Basic', 'count': 24},
             {'_id': 'Premium', 'count': 22},
             {'_id': 'VIP', 'count': 14}],
 'by_status': [{'_id': 'active', 'count': 50},
               {'_id': 'expired', 'count': 6},
               {'_id': 'frozen', 'count': 4}],
 'age_groups': [{'_id': 18, 'count': 12},
                {'_id': 25, 'count': 15},
                {'_id': 35, 'count': 22},
                {'_id': 45, 'count': 11}],
 'by_city': [{'_id': 'Chennai', 'count': 16},
             {'_id': 'Bengaluru', 'count': 13},
             {'_id': 'Hosur', 'count': 12},
             {'_id': 'Katpadi', 'count': 10},
             {'_id': 'Vellore', 'count': 9}]}

--- A7 Most prescribed exercises ($unwind embedded array)
exercise            plans  avg_sets  avg_weight_kg
------------------  -----  --------  -------------
Plank               23     4.0       0.0
Rowing Machine      20     3.7       0.0
Squat               19     4.2       67.6
Bench Press         17     4.0       55.9
Deadlift            17     3.9       74.4
Pull Ups            11     4.0       0.0
Treadmill Intervals 11     4.0       0.0
Barbell Row         10     3.6       39.5

--- A8 Peak check-in hours
hour   check_ins
-----  ---------
18:00  115
06:00  98
17:00  82
16:00  51
09:00  39
07:00  24

--- V1 member_dashboard view
{'member_id': 'MEM1005',
 'name': 'Rahul Rao',
 'tier': 'Premium',
 'status': 'expired',
 'trainer': 'Vivek Nair',
 'unpaid_amount': 0}
```

Screenshot 6: Aggregation pipelines A6 to A8 and the member_dashboard view (live MongoDB run)

5.2 Graph Traversal: Referral Network

Members can refer new members, who can refer others in turn, forming a tree of any depth. `$graphLookup` walks this tree recursively inside the database, which would otherwise need one query per level in application code. The `depthField` records each person's distance from the starting member, and `maxDepth` bounds the search. The `ix_referred_by` index supports each recursive step.

```
def referral_network(db, member_id):
    res = list(db.members.aggregate([
        {"$match": {"member_id": member_id}},
        {"$graphLookup": {"from": "members", "startWith": "$_id", "connectFromField": "_id",
                          "connectToField": "referred_by", "as": "network",
                          "maxDepth": 5, "depthField": "level"}},
        {"$project": {"_id": 0, "member_id": 1, "name": 1,
                      "network": {"$map": {"input": "$network", "as": "n",
                                           "in": {"member_id": "$$n.member_id", "name": "$$n.name",
                                                  "level": {"$add": ["$$n.level", 1]}}}}}},
    ]))
```

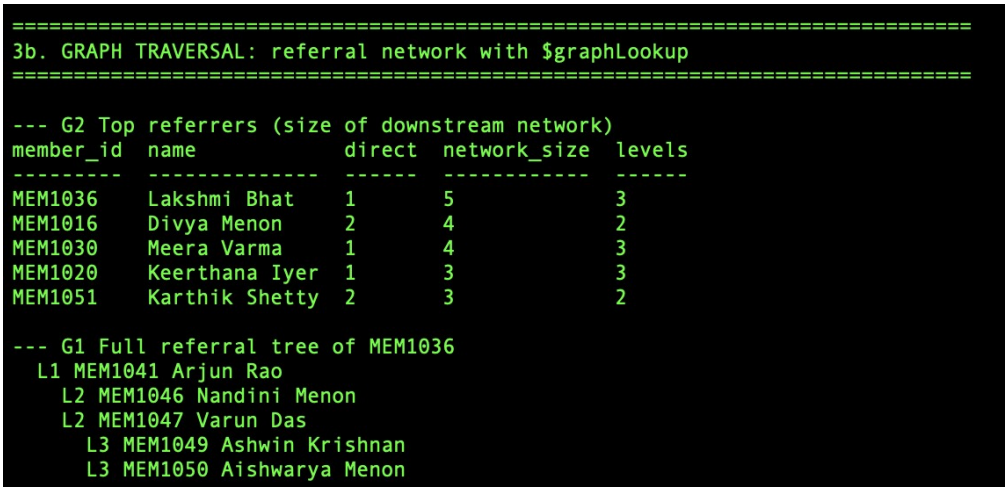
Member	Name	Direct referrals	Network size	Levels
MEM1036	Lakshmi Bhat	1	5	3
MEM1016	Divya Menon	2	4	2
MEM1030	Meera Varma	1	4	3
MEM1020	Keerthana Iyer	1	3	3
MEM1051	Karthik Shetty	2	3	2

G2 output: top referrers ranked by the size of their whole downstream network

Referral tree of MEM1036 Lakshmi Bhat (G1 output):

```
Level 1: MEM1041 Arjun Rao
    Level 2: MEM1046 Nandini Menon
    Level 2: MEM1047 Varun Das
        Level 3: MEM1049 Ashwin Krishnan
        Level 3: MEM1050 Aishwarya Menon
```

Lakshmi Bhat referred only 1 member directly, yet the traversal shows 5 members across 3 levels joined through that referral. Counting direct referrals alone would miss this, which is why graph traversal is the right tool for rewarding referrers.



Screenshot 7: $graphLookup referral network (live MongoDB run)

5.3 Indexing

Nineteen secondary indexes implement and extend the indexing plan from Review 1 section 6.8. Each index exists to serve a specific query in the CRUD layer or the reports.

Collection	Index	Keys	Type	Serves
members	uq_email, uq_member_id	email; member_id	Unique	Login, lookups, no duplicate accounts
members	ix_trainer	assigned_trainer_id	Single	Trainer's member list, A5
members	ix_tier_status	membership.type, membership.status	Compound	list_members filters
members	ix_referred_by	referred_by	Single	$graphLookup connectToField
trainers	uq_email, uq_trainer_id	email; trainer_id	Unique	Login, lookups
workout_plans	ix_member_status	member_id, status	Compound	get_active_plan
workout_plans	ix_trainer	trainer_id	Single	Trainer's plans, A5
classes	ix_trainer	trainer_id	Single	A5, delete_trainer check
classes	ix_enrolled_member	enrolled_members.member_id	Multikey	member_classes, rename propagation
classes	tx_class_search	class_name, category	Text	Class search
class_sessions	ix_class_date	class_id, date	Compound	Class timetable
attendance	ix_member_date	member_id, date (descending)	Compound	Member attendance history
attendance	uq_session_member	session_id, member_id	Unique compound	One record per member per session

Collection	Index	Keys	Type	Serves
attendance	ix_class_status	class_id, status	Compound	A2 class popularity
payments	ix_member_status	member_id, status	Compound	member_dues
payments	ix_due_date	due_date	Single	Overdue sweep
payments	px_unpaid_dues	status, due_date (unpaid only)	Partial	Dues reports on a smaller index

Proof with explain(). The same dashboard query, one member's attendance history newest first, is run twice: once forced to scan the collection with `hint({$natural: 1})`, and once through `ix_member_date`.

Plan	Leaf stage	Documents examined	Index keys examined	Documents returned	In-memory sort
Without index	COLLSCAN	580	0	5	Yes
With ix_member_date	IXSCAN	5	5	5	No

Without the index, MongoDB reads all 580 attendance documents and sorts them in memory to return 5. With the index it reads exactly the 5 matching entries, already in date order because the index stores `date` descending. As attendance grows over the years the scan cost grows with it, while the indexed query stays flat, which is how the design meets NFR1. The partial index `px_unpaid_dues` holds only the 67 unpaid bills instead of all 221, so dues reports use a much smaller index.

```
================================================================================
3c. INDEXING
================================================================================

--- Indexes on attendance and payments
  attendance._id_: [('_id', 1)]
  attendance.ix_member_date: [('member_id', 1), ('date', -1)]
  attendance.uq_session_member: [('session_id', 1), ('member_id', 1)] (unique)
  attendance.ix_class_status: [('class_id', 1), ('status', 1)]
  payments._id_: [('_id', 1)]
  payments.ix_member_status: [('member_id', 1), ('status', 1)]
  payments.ix_due_date: [('due_date', 1)]
  payments.px_unpaid_dues: [('status', 1), ('due_date', 1)] (partial)

--- explain(): member attendance history, collection scan vs index scan
plan           stage     docs_examined  keys_examined  returned  time_ms
-------------  --------  -------------  -------------  --------  -------
without_index  COLLSCAN  580            0              5         0
with_index     IXSCAN    5              5              5         0

--- Text index search: classes matching 'yoga'
class_name    category
------------  --------
Evening Yoga  Yoga
Morning Yoga  Yoga
```

Screenshot 8: Indexes, explain() comparison and text search (live MongoDB run)

5.4 Multi-Document ACID Transactions

Two operations write to more than one collection and must never be left half done. They run inside a transaction with snapshot read concern and majority write concern. PyMongo's `with_transaction` retries transient errors automatically and aborts on any exception, so either every write commits or none does.

Transaction	Writes	Why it must be atomic
T1 renew_membership	Insert a paid bill in payments; extend end_date in members	A payment without an extension, or the reverse, is a billing error

Transaction	Writes	Why it must be atomic
T2 close_session	Mark the session completed; insert its attendance rows	Attendance must never exist for a session still marked open, and a failed roll call must not half-save

```
def _run(db, fn):
    """Run fn(session) inside a transaction, with automatic retry on transient errors.
    Under mongomock (no transaction support) fn runs without a session."""
    if is_mock():
        return fn(None)
    client = get_client()
    with client.start_session() as s:
        return s.with_transaction(
            fn,
            read_concern=ReadConcern("snapshot"),
            write_concern=WriteConcern("majority"),
            read_preference=ReadPreference.PRIMARY,
        )

def renew_membership(db, member_id, months, mode="UPI", fail_midway=False):
    """T1. fail_midway=True raises after the payment insert to prove the rollback."""
    def txn(s):
        m = db.members.find_one({"member_id": member_id}, session=s)
        if not m:
            raise TransactionAborted(f"{member_id} not found")
        fee = TIER_FEES[m["membership"]["type"]] * months
        old_end = m["membership"]["end_date"]
        start = max(old_end, REFERENCE_DATE) + timedelta(days=1)
        new_end = start + timedelta(days=30 * months)
        last = db.payments.find_one({}, sort=[("payment_id", -1)], projection={"payment_id": 1},
session=s)
        pid = f"PAY{int(last['payment_id'][3:]) + 1}"
        db.payments.insert_one({
            "payment_id": pid, "member_id": m["_id"], "member_name": m["name"], "amount": fee,
            "currency": "INR", "payment_mode": mode, "status": "paid",
            "billing_period_start": start, "billing_period_end": new_end,
            "payment_date": REFERENCE_DATE, "due_date": REFERENCE_DATE}, session=s)
        if fail_midway:
            raise TransactionAborted("Simulated failure after payment insert")
        db.members.update_one({"_id": m["_id"]},
                              {"$set": {"membership.end_date": new_end, "membership.status": "active"}},
                              session=s)
        return {"payment_id": pid, "amount": fee, "new_end_date": new_end.date().isoformat()}
    return _run(db, txn)
```

Test	Result
T1 with a simulated failure after the payment insert	Aborted; payment count and end date unchanged (rolled back)
T1 renew MEM1010 for 3 months	Committed: new paid bill of INR 4,500; end date moved from 31 Mar 2027 to 30 Jun 2027
T2 close the next session with 3 roll-call marks	Committed: session completed, 3 attendance rows added
T2 with a member who is not on the roster	Aborted after the session update and first insert; both rolled back, session stays scheduled

```
================================================================================
3d. MULTI-DOCUMENT ACID TRANSACTIONS
================================================================================

--- T1 renew_membership with a simulated failure -> rollback
aborted: Simulated failure after payment insert
payments count 220 -> 220 (unchanged) | end_date still 2027-03-31

--- T1 renew_membership, 3 months -> commit
{'payment_id': 'PAY7221', 'amount': 4500, 'new_end_date': '2027-06-30'}
end_date 2027-03-31 -> 2027-06-30 | payments 220 -> 221

--- T2 close_session: attendance rows + session status in one transaction
SES9006 Morning Yoga 2026-09-24: {'session_id': 'SES9006', 'attendance_rows': 3}
session status now: completed

--- T2 with a member not on the roster -> whole transaction rolled back
aborted: MEM1001 is not enrolled in Pilates Core
attendance rows 583 -> 583 | SES9046 status: scheduled

Demo complete.
```

Screenshot 9: Transaction commits and rollbacks (live MongoDB run)

6. Traceability to Review 1 Requirements

ID	Requirement	Status in Review 2	Where
FR1	Admin CRUD on members, trainers, classes, payments	Done	crud.py
FR2	Trainer manages workout plans	Done	plan functions; A5, A7
FR3	Member views profile, plan, classes, attendance, payments	Done at data layer; screens in Review 3	member_dashboard view, member_classes, attendance_history, member_dues
FR4	Class scheduling with capacity and enrollment	Done	create_class, create_session, enroll_member, withdraw_member
FR5	Attendance per member per session	Done	mark_attendance, T2 close_session
FR6	Payment tracking and overdue flags	Done	create_invoice, record_payment, flag_overdue, A4
FR7	Aggregated reports	Done	A1 to A8
FR8	Authentication and role-based access	Authentication done; role-based screens in Review 3	authenticate (bcrypt)
NFR1	Performance	Done	19 indexes, explain() comparison
NFR2	Scalability	Done	Unbounded data referenced, bounded data embedded; indexed access paths
NFR3	Security	Done	bcrypt hashes; projections hide password_hash
NFR4	Usability	Review 3	Responsive dashboard
NFR5	Data consistency	Done	Validators, 10 integrity rules, atomic updates, transactions

7. Testing

Six automated tests in `tests/test_gym_db.py` run against an in-memory MongoDB (mongomock), so any team member or evaluator can verify the code without installing a server. All six pass.

Test	Checks
test_minimum_sample_records	At least 50 members and 1,000 documents are loaded
test_member_crud_and_cascade	Create, read, rename propagation, enrollment and cascading delete leave no orphans

Test	Checks
test_integrity_guards	Unknown trainer rejected; a trainer with members cannot be deleted
test_capacity_guard	A class of capacity 1 rejects a second member and a duplicate enrollment
test_payments_flow	Overdue sweep flags bills; a payment is recorded once; a paid bill cannot be paid again
test_aggregations	Revenue report equals the sum of paid bills; facet totals equal the member count; graph sizes agree

In addition, all 1,016 seeded documents were checked against the seven `$jsonSchema` validators before submission, with zero violations. Finally, the complete demonstration was run against a live MongoDB 8.3 replica set: validators, explain(), text search and both transaction rollbacks behaved exactly as designed. Screenshots 1 to 9 are taken from that run.

8. Intermediate Demonstration Plan

The demonstration takes about ten minutes and follows the rubric order. `demo.py` recreates the database at the start, so the panel always sees the same state.

Step	Command	What the panel sees
1	python demo.py --section setup	Collections with validators, 1,016 documents, 19 indexes, a validator rejecting bad data
2	python demo.py --section crud	Every CRUD operation, including the blocked cases in Section 4.4
3	python demo.py --section agg	Reports A1 to A8 and the member_dashboard view
4	python demo.py --section graph	Top referrers and a full referral tree
5	python demo.py --section index	Index list, COLLSCAN versus IXSCAN, text search
6	python demo.py --section txn	Commit and rollback of both transactions
7	mongosh ... mongosh/demo_queries.js	Any query re-run live in the shell or Compass on request

9. Source Code Repository

Repository: https://github.com/malikabalaji/gym-membership-db

The repository holds the complete source code, sample data, tests, the live demonstration output (`demo_output.txt`), the Review 1 and Review 2 reports, and the terminal screenshots from the live run.

```
gym-membership-db/
  README.md                 setup, run commands, dataset summary
  requirements.txt          pymongo, bcrypt, mongomock, pytest
  demo.py                   intermediate demonstration (6 sections)
  export_data.py            exports the dataset to data/*.json
  gym_db/
    config.py               connection, reference date, tier fees
    schema.py               collections and $jsonSchema validators
    seed.py                 reproducible sample data (1,016 documents)
    indexes.py              19 indexes and the explain() comparison
    crud.py                 36 CRUD functions and integrity rules
    aggregations.py         A1 to A8, G1 and G2 ($graphLookup), view V1
    transactions.py         T1 and T2 ACID transactions
    mockpatch.py            offline test support
  mongosh/demo_queries.js   live shell queries
  data/*.json               exported sample data (Extended JSON)
  tests/test_gym_db.py      6 automated tests
```

Running it:

```
pip install -r requirements.txt
docker run -d --name gymdb -p 27017:27017 mongo:8 --replSet rs0
docker exec gymdb mongosh --eval "rs.initiate()"
python demo.py              # full demonstration
python demo.py --mock       # same demonstration without a server
python -m pytest -q         # automated tests
```

10. Plan for Review 3

- Web dashboard for the three roles (Admin, Trainer, Member), built on the CRUD layer and the member_dashboard view.
- Session-based login with role-based authorization, completing FR8.
- Admin dashboard charts fed directly by pipelines A1 to A8.
- Responsive layout for desktop and mobile, completing NFR4.